

Концептуальное Руководство

Лэнгсмит

Агентам нужен собственный компьютер. Вот как безопасно его предоставить.



Эми Ру

15 июля 2026 года



12 мин

[← Вернуться к блогу](#)

[Зачем агентам нужен собственный компьютер](#)

[Почему нельзя просто передать ему свою инфраструктуру](#)

[Четыре функции, которые должен хорошо выполнять компьютер любого агента](#)

[Когда лучше использовать управляемую песочницу, а когда – создавать собственную](#)

[LangSmith Sandboxes: компьютер для каждого агента](#)

[Примечание о промпт-инъекциях в рабочих процессах в изолированной среде](#)

[Варианты использования](#)

[Заключение](#)



Поделиться

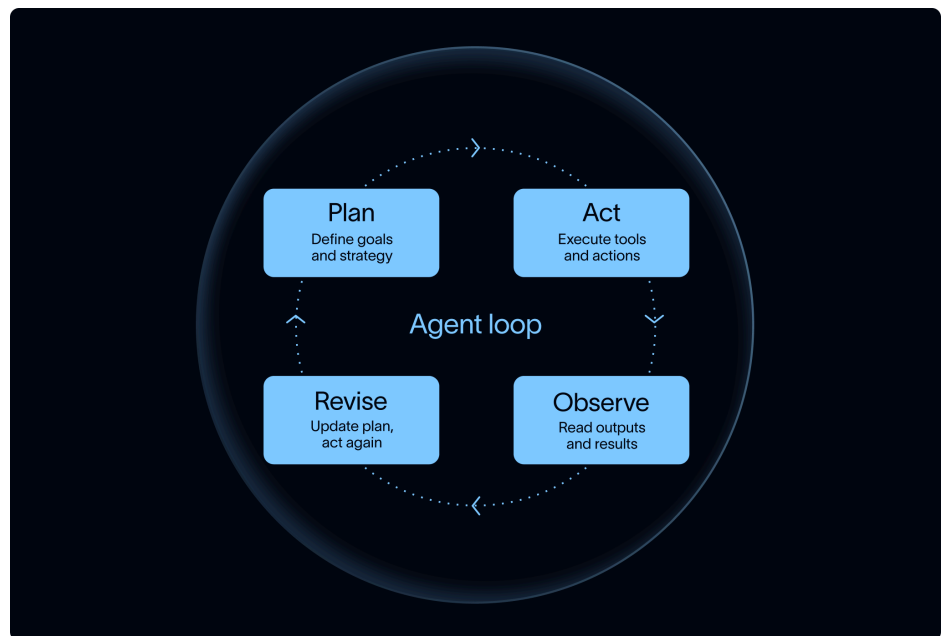


Почему агентам нужен собственный компьютер

Попросите большую языковую модель отладить неудачный тест или очистить набор данных, и она сделает большую часть работы. Она может объяснить, как исправить ошибку, написать запрос и провести анализ. Но на этом она останавливается, и вам приходится доделывать все остальное.

Проблема в том, что система, которая может генерировать только текст, подобна подрядчику, который может в точности описать, как починить вашу сантехнику, но у него нет ни рук, ни инструментов, ни грузовика. Совет может быть идеальным, но кто-то все равно должен взять в руки гаечный ключ.

Агенты устраняют этот пробел, используя руки. Если дать модели возможность запускать код, считывать результат и повторять попытку, то полный цикл работы агента позволит ему действовать более автономно:



Агент, который может только предложить решение, не может знать, сработает ли оно. Вот почему агентам нужен собственный компьютер: реальная среда с файловой системой, оболочкой, менеджером пакетов, доступом к сети и состоянием, которое сохраняется на протяжении всех шагов.

И если у вас один ноутбук и вы, возможно, единственный, кто им пользуется, то агентская платформа может параллельно запускать тысячи таких сред, каждая из которых должна быть изолированной, одноразовой и безопасной для передачи реальной вычислительной мощности.

Что это значит на практике?

- Агент по написанию кода может клонировать репозиторий, установить зависимости, запустить набор тестов, проанализировать ошибки, внести исправления в код и вернуть diff-файл, который уже прошел проверку.
- Агент по анализу данных может загрузить необработанные файлы, проверить схему, написать код для преобразования на Python или SQL, сгенерировать диаграмму и проверить собственные вычисления, прежде чем показать вам отчет.
- Агент по проведению исследований может просматривать веб-страницы, скачивать источники, анализировать и нормализовать их, сопоставлять утверждения и составлять итоговый отчет с цитатами.

В каждом примере модель выполняет реальные действия и проверяет реальные результаты. Для этого нужно рабочее пространство, а не просто контекстное окно для рассуждений.

Почему вы не можете просто передать ему свою инфраструктуру

Так почему бы просто не позволить агенту запускать код локально или в контейнере Docker?

Большинство прототипов разработчиков работает довольно быстро.



Products ▾ Learn ▾ Docs Company ▾ Pricing Try LangSmith

Get a demo

Но эта же схема начинает давать сбои по двум конкретным причинам.

- **Безопасность:** Код, который собирается запустить ваш агент, мог быть написан, проверен или даже просмотрен человеком до того, как он был запущен.
- **Изоляция:** Стандартные границы контейнера не предназначены для защиты от ненадежных действий, вызванных использованием моделей.

И хотя у вашего агента нет злого умысла, вы не знаете, откуда берется этот код. Строка кода может быть взята из самой модели, из клонированного репозитория или из пакета, установленного в процессе работы. Например, исследовательский агент анализирует документы, которые он скачал из интернета. Код, выполняемый агентом, может быть сгенерирован за несколько секунд до запуска, сформирован непосредственно на основе того, что ввел пользователь, и создан в процессе выполнения задачи. Между этими этапами нет этапа проверки.

A well-written prompt doesn't give you immunity from security concerns. The safest posture is machine-level separation: give the agent a real environment to work in, but keep that environment isolated from your laptop, from production, and from every other agent's workspace running alongside it.

Four things every agent's computer needs to do well

Four things every agent's computer needs to do well

01

Execute

Execute safely

Run untrusted code without risk to the host system.

02

Control

Stay in control

Enforce boundaries, permissions, and resource limits.

03

Observe

Be observable

Full visibility into what ran, when, and what changed.

04

Build

Build and iterate fast

Spin up environments quickly, ship and improve with speed.

1. Execute safely

In 2025, a self-replicating npm worm backdoored hundreds of packages and executed in preinstall hooks before any tests ran. A Linux kernel CVE disclosed in 2026 could root any major distribution with a 732-byte Python script in about an hour, and containers couldn't help, because they shared a kernel with the host.

Agent-executed code should be treated as untrusted by default, regardless of its source. This includes code the model wrote, code pulled from a cloned repository, packages installed mid-task, and scripts produced by multi-step reasoning chains. Each agent workspace should be a hardware-virtualized machine with its own kernel, filesystem, and network boundary.

2. Stay in control

Controls inside a sandbox protect you from the agent doing expensive, unexpected, or credential-leaking things.

- **Credential management:** Agents often need to call external services (i.e. APIs, databases, storage) to do their work. In a sandbox, you can route outbound requests through a proxy that injects credentials at the network layer, which means the agent can make the call without ever seeing the token.
- **Resource limits:** An agent running in a loop can consume a surprising amount of compute and network. CPU limits, memory caps, and network allowlists/denylists let you define a cost ceiling per task and prevent runaway execution.
- **Lifecycle controls:** Sandboxes are ephemeral by default. They spin up for a task, persist state while needed, and get cleaned up when idle.

3. Be observable

Observability in agent execution means knowing:

- Which commands ran

- Which files were created or modified
- Which network calls went out
- Which packages were installed
- What the output was at each step

Essentially, this is an audit log for workflows, especially ones that touch sensitive data or take actions with real-world consequences. What makes an agent reliable is the ability to re-run from a known state, compare branches, and trace what actually happened.

4. Build and iterate fast

Production requirements need to be fast provisioning (sub-second when warm), reproducible environments (defined by a Docker image or blueprint that every instance starts from), and persistent state (files, installed packages, and session context carry over between agent turns). If spinning up an execution environment takes thirty seconds, agents that need multiple environments in a task will feel slow. If environments aren't reproducible, bugs become hard to isolate. If state doesn't persist across sessions, long-running tasks require expensive restarts.

When to reach for a managed sandbox vs. building your own

You can approach this in a DIY fashion: run the agent on a developer's laptop, graduate to a Docker container for some separation, wire up resource limits and credential injection manually. For agents that only call external APIs with fixed schemas and never execute dynamic code, this is often enough.

When you want to scale to generating scripts, installing packages, running test suites, or parsing files, you'll have to:

- Spin up VMs per task
- Inject credentials securely without touching the runtime
- Build snapshot and fork support for parallel branches
- Log execution traces and tie them to agent traces
- Handle teardown, idle detection, and quota management

Here's the decision criteria:

- If your agent only calls APIs and executes no dynamic code, local or containerized execution is likely fine.
- If your agent executes model-generated code, installs packages, or processes arbitrary files, you'll need real isolation, and building it from scratch means you're building a sandbox platform.

The operational overhead of a DIY approach for production agent deployments adds up fast. The managed sandbox path trades that engineering surface for a simpler interface with a platform that handles the scale of work.

LangSmith Sandboxes: a computer for every agent



Each LangSmith sandbox boots fast (median under one second), runs as a hardware-virtualized microVM with its own kernel, and persists state (files, installed packages, environment) across the agent's working session. When the task is done, the sandbox idles and gets cleaned up automatically.

While a container shares a kernel with the host, a microVM has its own. Inside the sandbox, the agent can install anything, run Docker, start services — all while your infrastructure and other workloads stay untouched.

Core primitives

- **MicroVM isolation:** Every sandbox is kernel-isolated from your services and from other sandboxes. What runs in one can't affect what runs in another.
- **Snapshots and forks:** Capture the state of a running sandbox, or build one from a Docker image. Forks are copy-on-write, so spinning up ten parallel branches from the same snapshot costs roughly the same as one. When an agent goes down a wrong path, you can restore to the snapshot and try a different branch.
- **Auth Proxy:** Outbound requests from a sandbox flow through a proxy that injects credentials at the network layer. The agent can call GitHub, an S3 bucket, or an internal API, and the secrets never touch the runtime. Domain allowlisting gives you control over what the sandbox can reach.

- **Service URLs:** You'll get authenticated HTTP access to anything running inside a sandbox, whether a preview server, a local API, or a database. Share the URL with a teammate and open it in a browser. No port forwarding needed.

One-call setup

```
1 `from langsmith.sandbox import SandboxClient`  
2  
3 `client = SandboxClient()`  
4  
5 `with client.sandbox() as sb:`  
6  
7 `result = sb.run("python my_analysis.py")`  
8  
9 `print(result.stdout)`
```

Sandboxes work with Deep Agents, Open SWE, LangSmith Deployment, LangSmith Fleet, and any custom code. They use the same SDK and API key as the rest of LangSmith.

A note on prompt injection in sandbox workflows

Sandboxes provide strong execution isolation, but they don't change a fundamental property of language models: anything the agent reads can influence what the agent does next. This matters when sandbox output is fed back into the model.

An example concern is a research agent downloads a document, the document contains text designed to look like an instruction to the model, and the model follows it. This is the #1 vulnerability in the [OWASP Top 10 for LLM Applications](#), and it applies to any agent that processes external content, whether that's web pages, uploaded files, API responses, or code execution output.

Sandboxes don't eliminate the threat of injection, but they do contain the execution blast radius. Malicious code that runs inside a sandbox can't reach your host, but if the output of that execution is read back by the agent without scrutiny, an injected instruction in the output can still influence downstream behavior.

How you can mitigate:

- Treat sandbox output as untrusted data. The model should not act on imperative text found in execution output without confirmation.
- For the most sensitive workflows, use a "non-agentic read" pattern: have a non-model process retrieve the finished artifact from the sandbox (a file, a

diff, a report), rather than routing raw output through the agent's context.

- Limit what a cross-boundary agent can access locally. If the agent operates on both local and sandbox environments, apply least-privilege. The local access surface should be as narrow as the task requires.
- Apply output filtering at the boundary, e.g. structured output schemas, content classifiers, or format validation reduce the attack surface without requiring the model to resist injection on its own.
- Don't rely on prompting the model to detect or ignore injections. Adversarial research consistently shows this is insufficient at scale.

None of this is unique to sandboxes. Any agent that reads from the web, processes uploaded files, or calls external APIs has the same exposure. The added value of Sandboxes is that they help you contain the execution damage, and both layers are needed.

Use cases

Coding agents

Coding agents are a popular use case for sandboxes since execution isolation matters a lot. An agent that can run a test suite, inspect the failure, patch the code, and run the suite again is qualitatively more useful than one that can only generate a patch and hand it back for the developer to validate.

Coding agents in sandboxes do things like:

- Clone a repository and run its full setup script, including npm install, pip install, or whatever the project requires, without those packages touching the developer's machine.
- Run the test suite against the current state of the code, read failure output, form a hypothesis, and iterate until tests pass.
- Open a PR with a diff that has already been verified to pass CI.
- Run in ten different fix attempts in parallel from a snapshot, returning the one that worked.

For sandbox environments, the snapshot-and-fork pattern has worked especially well for CI-style agents like [Open SWE](#). A single snapshot captures the repo and installed dependencies, then each candidate fix runs in its own fork, with the successful fork's diff surfaced as the result.

Data analysis agents

Data analysis agents operate over files and databases that are often sensitive (financial records, health data, customer data, risk models). Data sensitivity and model-generated code means execution isolation is important.

A data analysis agent in a sandbox can:

- Load a file upload (CSV, Excel) into a clean environment where only the analysis code can touch it.
- Write and run Python, SQL, or R to clean, transform, and analyze the data, with errors fed back to the model for correction without any of that I/O leaving the sandbox.
- Generate charts, validate calculations, and produce a finished report with outputs that are retrieved from the sandbox as finished artifacts, not streamed through the agent's context.

Use cases in this category include analytics agents, financial research and underwriting, insurance claims processing, and scientific data pipelines — any time when sensitive data makes running code in your production environment a security or compliance concern.

Research agents

Research agents span the most varied execution surface. A deep research workflow might browse the web, download PDFs, run a scraper, call a data API, normalize multimedia content, and synthesize a structured output.

What sandboxes give research agents:

- A real browser and network environment for browsing and scraping, isolated from your infrastructure.
- A filesystem for staging downloaded files, parsed documents, and intermediate outputs.
- A compute environment for running normalization scripts, calling APIs, and generating output formats (i.e. charts, audio summaries, slide decks).
- A clean teardown when the task is done, with the finished artifact retrieved and everything else discarded.

Use cases in this category include competitive intelligence, investment due diligence, academic research pipelines, and any workflow that requires synthesizing information from multiple external sources into a finished, structured output.

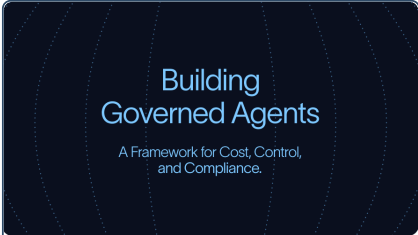
Conclusion

For most of computing history, a developer environment meant a physical machine, then a VM, then a container, with each one shared across the work happening on it, each one requiring deliberate setup and teardown. The idea of giving every agent its own isolated computer, booting in under a second and cleaning up when done wasn't practical at scale.

Sandboxes give you a safer way to run upgraded workflows, providing you an isolated environment to run tasks that require iteration, verification, and access to the tools a person would normally use, all without requiring human supervision.

Every human developer gets a laptop. Every agent can get a computer.

Related content



Building Governed Agents
A Framework for Cost, Control, and Compliance.

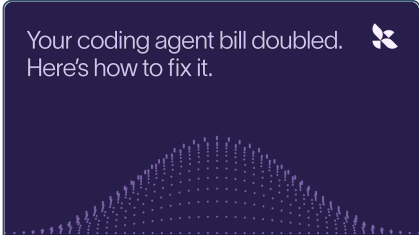
Conceptual Guide

LangSmith

Building Governed Agents: A Framework for Cost, Control, and Compliance

 Martha Janicki
July 20, 2026

🕒 15 min



Your coding agent bill doubled. Here's how to fix it.

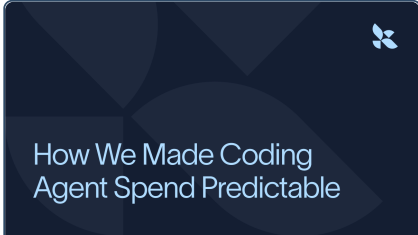
LangChain LangSmith

Observability & Evals

Your coding agent bill doubled. Here's how to fix it.

 Amy Ru
July 2, 2026

🕒 6 min



How We Made Coding Agent Spend Predictable

LangSmith

How We Made Coding Agent Spend Predictable

 Martha Janicki
June 15, 2026

🕒 5 min



Sign up for our newsletter to stay up to date

joh

Subscribe

See what your agent is really doing

LangSmith, our agent engineering platform, helps developers debug every agent decision, eval changes, and deploy in one click.

Try LangSmith

Get a demo

Products

LangSmith Platform
LangSmith Observability
LangSmith Evaluation
LangSmith Deployment
LangSmith Fleet
LangSmith Sandboxes
Deep Agents
LangChain
LangGraph

Resources

Blog
Customer Stories
Guides
Community
Changelog
Docs
Support
LangChain Academy

Company

About
Careers
Partners
Trust Center
Marketing Assets
Events

Sign up for our newsletter to stay up to date

Your email

Subscribe



LangChain

All systems operational

Privacy policy

Terms of service